



Material de Clase 03

PROGRAMACIÓN EXTREMA (XP)

La programación extrema tiene una gran influencia en el funcionamiento actual de las industrias de software. TDD, programación en pares, revisiones de código se han convertido en los estándares de facto como parte de XP.

Incluso después de 30 años de su creación, la programación extrema sigue siendo relevante y Martin Fowler llama a la programación extrema la base de las prácticas ágiles.

Exploremos cómo y por qué...

En los primeros días de la programación, los proyectos de software se ejecutaban solo con las mejores habilidades de las personas que los escribían. Un pequeño error y el código causa estragos.

Incluso sin errores, una vez que la base de código se vuelve lo suficientemente grande, es difícil mantener el código existente, y mucho menos agregar nuevas características.

Los programadores se esfuerzan al máximo para asegurarse que el código siga funcionando, cumpliendo los plazos, etc., mientras que los clientes/partes interesadas probablemente se preguntaron si los programadores están siendo perezosos y no cumplen.

Muchos de estos problemas se volvieron más fáciles de resolver y los esfuerzos para hacerlo se redujeron significativamente. Llegó con algo llamado **programación extrema**.

Veamos cómo funciona y por qué Kent Beck pensó en él y cómo incluso después de 30 años de su creación sigue siendo relevante.

¿Qué es la programación extrema y cómo resolvió todo?

La programación extrema es una práctica basada en Agile que básicamente garantiza un software de la mejor calidad en poco tiempo, teniendo en cuenta el bienestar de los programadores y el compromiso con los clientes/accionistas.

Historia

Todo comenzó en C3 (proyecto de Compensación Integral de Chrysler), un proyecto de nómina en Chrysler que se considera el lugar de nacimiento de la Programación Extrema.

C3 tenía como objetivo reemplazar el sistema basado en COBOL heredado y el desarrollo inicial comenzó en SmallTalk, que se suponía que terminaría en 2 años. Un año y medio después... hay algunos problemas de rendimiento y se supone que Kent Beck los ayudará a resolverlos.

Kent Beck: "*¿Dónde están las pruebas que demostrarán que cuando hago las cosas más rápido, nada se rompe?*"

C3: "En realidad, el código aún no funciona"

Kent Beck sugiere 3 opciones:

- Sigue todo como está.
- Subcontratar el proyecto (lo cual es una pena con gente tan excepcional, Martin Fowler también trabajó como consultor en este proyecto)
- Dé a todos una semana libre, deseche todo lo que se ha hecho hasta ahora y comience de nuevo y creo que tendrá éxito.

Kent Beck se pone a cargo. Él estructura el proyecto en incrementos de 3 semanas donde el trabajo se divide en historias y al final de cada incremento una demostración para los usuarios y un plan para las próximas 3 semanas.

Luego, alguien sugiere que cada historia debería tener alguna prueba de aceptación. Entonces alguien sugiere programación en pareja, etc. Y así sucesivamente, al final del día, había formulado una programación extrema. Más adelante en su libro, Kent Beck estandariza esta forma de hacer las cosas y la llama **programación extrema**.

Comprender por qué funciona la programación extrema

La programación extrema no es más que un conjunto de valores, principios y prácticas.

Eso es todo.

¿Por qué? Mejorar la calidad del código y del producto teniendo en cuenta el bienestar de los desarrolladores.

Para elaborar una programación más extrema se adhiere a ciertos valores no negociables (que sabemos que producirán buena calidad). Para adherirnos a estos valores utilizamos ciertas prácticas y mientras usamos estas prácticas, tenemos ciertos principios/directrices que debemos seguir para lograr los mejores resultados posibles.

Para explicarlo mejor, para un atleta el valor es tener un gran cuerpo, practica: haz ejercicio diariamente y el principio es no usar esteroides.

¿Cómo garantizarían estos valores una mejor calidad del código y el bienestar del desarrollador?, podría preguntarse.

Exploremos uno por uno.

- **Comunicación:** como todo lo demás, para poder lograr los mejores resultados, todos deben estar en la misma página. Los equipos deben hablar, explicarse y entenderse entre ellos.
- **Comentarios:** una vez que tengamos la misma dirección de vez en cuando, debemos reevaluarnos a nosotros mismos a partir de los comentarios brindados por los clientes o miembros de su equipo. Esto asegura que nos dirigimos hacia el mejor resultado posible con la satisfacción personal y del cliente.
- **Simplicidad:** Las soluciones más elegantes no son las que son complejas sino las que son sencillas de entender.
- *"Es mejor hacer algo simple hoy y pagar un poco más mañana para cambiarlo" que "hacer algo más complicado hoy que quizás nunca se use de todos modos".*

- Esto significa menos explicaciones al incorporar nuevos miembros y una fácil mantenibilidad de nuestro trabajo existente.
- **Valentía:** Expresar sus opiniones, discutir y tomar decisiones por el bien del equipo/producto/cliente. A menudo conduce a las mejores ideas que originalmente no formaban parte del plan o ni siquiera se consideraron.
- **Respeto:** estar abierto a los demás y considerar la opinión y los pensamientos de todos no solo anima a las personas a compartir grandes ideas, sino que también se asegura de que los demás lo escuchen.

Ahora que tenemos nuestros valores establecidos, exploremos los principios.

Principios

Principios: retroalimentación rápida, simplicidad, cambio incremental, aceptación del cambio y calidad

Retroalimentación rápida: obtenga retroalimentación con frecuencia para asegurarse de que está satisfaciendo las necesidades del cliente y reflejando cuando ha aprendido a través de la retroalimentación en el producto y su trabajo.

Simplicidad: Resuelve cada problema de la forma más sencilla. Haga solo lo necesario para implementar realmente la función. Cualquier cosa extra puede agregar complejidad y puede ser completamente obsoleta y difícil de cambiar más tarde.

Sigue a YAGNI (No lo vas a necesitar) y DRY (No te repitas)

Cambio incremental: tratar de hacer muchas cosas a la vez puede terminar introduciendo muchos errores, código no deseado y terminar sin funcionar a tiempo. Los grandes cambios a la vez están destinados a fracasar. Por lo tanto, cada cambio debe desglosarse en una serie de pequeños pasos. Esto ayuda a rastrear el progreso y las unidades pequeñas también se pueden probar de manera eficiente.

Esta filosofía se puede aplicar al proyecto, al diseño, a la arquitectura o también a los cambios en la plantilla.

Aceptar el cambio: se deben aceptar los cambios en el diseño, el proyecto y el plan. Esto podría dar una ventaja competitiva al cliente para realizar cualquier cambio con confianza en función del mercado/entorno y la tranquilidad general para que el equipo sea lo suficientemente competente como para aceptarlo.

Trabajo de calidad: Un equipo que trabaja bien, hace un producto valioso y se siente orgulloso de ello. El cliente en general tiene una gran puntuación de satisfacción al final.

Ahora veamos algunas prácticas que nos ayudan a adherirnos a nuestros valores teniendo en cuenta nuestros principios.

TDD: el desarrollo basado en pruebas es una práctica en la que primero escribe las pruebas y luego escribe el código. Esto asegura una retroalimentación inmediata. También mantiene el código nítido y simple, ya que cada vez que escribe solo escribe tanto código como sea necesario para pasar la prueba. Cuando haya terminado, también tendrá una red de seguridad para continuar refactorizando sin romper nada.

Refactorización de código: la refactorización nos permite mejorar continuamente el código. Ayuda a eliminar la redundancia, el código innecesario y aumenta la legibilidad. Mantener el código nítido y simple nos permite realizar cambios fácilmente incluso después de mucho tiempo.

CI/CD: Permite que el equipo colabore en diferentes partes del código sin depender de nadie. El código es enviado varias veces al día por diferentes miembros. Las pruebas automatizadas y de extremo a extremo aseguran que todo funcione correctamente.

Pequeñas liberaciones-CI y pequeñas liberaciones van de la mano. CI nos permite lanzar versiones incrementales del producto para recibir comentarios del cliente. Esto nos permite detectar errores y probar el código en producción. El cliente puede sugerir un cambio anticipado después de cada lanzamiento.

Programación en pareja: es una técnica en la que dos desarrolladores se sientan juntos y escriben código. Mientras uno escribe, los otros revisan el código, sugieren mejoras y buscan errores. Luego se intercambian los roles. Esto significa un intercambio de conocimientos más rápido y una mejor comunicación. Los desarrolladores tienden a aprender prácticas unos de otros, lo que ayuda a establecer el código estándar a nivel de equipo/organización.

Revisiones de código: se basan en el modelo de relaciones públicas en el que los desarrolladores generan una relación pública para cada historia/función en la que trabajan. Esto permite adherirse al estándar de código de nivel de organización. Se puede recomendar a los desarrolladores que realicen cambios, si los hubiera. Esto también asegura que haya un segundo par de ojos que escanee el código asegurándose de que haya suficiente cobertura de código, que no haya errores y que, en general, la calidad del código sea buena.

Las herramientas automatizadas, como la nube SONAR, ayudan en las revisiones de código.

Semana de 40 horas: Esto asegura el bienestar del desarrollador. Están bien descansados y en su mejor momento cuando trabajan. Un equipo es tan bueno como sus miembros. Esto garantiza que no haya agotamiento y que los desarrolladores puedan mantener un buen equilibrio entre el trabajo y la vida.

Cliente en el sitio: el cliente debe ser parte del proceso de desarrollo para responder cualquier pregunta, establecer prioridades y resolver disputas, si las hubiera.

El juego de planificación: esta es una reunión que ocurre al comienzo de un ciclo de iteración. El equipo de desarrollo y el cliente se reúnen para discutir y aprobar las características de un producto. Al final del juego de planificación, los desarrolladores planifican la próxima iteración y el lanzamiento, asignando tareas para cada uno de ellos.

Estándares de codificación: tener un conjunto común de prácticas de codificación, usar los mismos formatos y estilos para la escritura de código permite a todos los miembros del equipo leer, compartir y refactorizar el código con facilidad, así como acelerar el aprendizaje para otros programadores.

Propiedad colectiva del código: esta práctica delega la responsabilidad del código a todo el equipo, donde cada miembro es igualmente responsable. Esto garantiza que todo el equipo conozca todo el código y sepa exactamente dónde colocar una determinada característica y evitar la duplicación.

Metáfora del sistema: la metáfora del sistema representa un diseño simple de modo que las personas nuevas puedan comenzar a trabajar en él sin perder demasiado tiempo examinando las especificaciones. Y en general, el código es legible.



Bibliografía y recursos

En el siguiente link, encontrarás material que recomendamos leer: Comparación entre Scrum y XP
(<https://betterprogramming.pub/a-comparison-between-scrum-and-extreme-programming-7c3fb7b63df6>).